Executive Summary

The dvGuidance module produces an inertially fixed attitude reference frame $\mathcal{R}$ for a delta-V burn. The burn frame is constructed from the commanded delta-V vector and a preferred direction for its 2nd axis: the 3rd axis points along the commanded delta-V vector, and the 2nd axis points as close as possible to the preferred direction. The output is the MRP attitude $\boldsymbol{\sigma}_{R/N}$ of $\mathcal{R}$ with respect to the inertial frame $\mathcal{N}$. Because $\mathcal{R}$ is inertially fixed, the published reference angular rate $\boldsymbol{\omega}_{R/N}$ and angular acceleration $\dot{\boldsymbol{\omega}}_{R/N}$ are zero.

The module does not rotate the burn frame. When a rotation is required, the output of dvGuidance is used as the base frame $\mathcal{R}_0$ of the `mrpRotation` module, which applies the rotation to produce $\mathcal{R}$.

This module derives from the Xmera `dvAttGuidance` module. Unlike the original, the burn frame is inertially fixed, and the commanded delta-V vector and preferred 2nd-axis direction are configuration parameters instead of inputs read from a burn command message. Inputs and outputs are single-precision (FP32); the algorithm is single-precision throughout.

Module Architecture

The module is split into a thin adapter (`DvGuidance`) that handles framework integration and an algorithm class (`DvGuidanceAlgorithm`) that contains the pure math.

Adapter Layer

The adapter inherits from `SysModel` and owns the output message. It follows the two-phase init pattern: the public configuration properties are set after construction, and `reset()` validates them and constructs the algorithm. Each `updateState()` publishes the algorithm's $\boldsymbol{\sigma}_{R/N}$ with zero reference rate and acceleration.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
`attRefOutMsg`	`AttRefMsgF32Payload`	Reference attitude / rate / acceleration of $\mathcal{R}$ relative to $\mathcal{N}$.

Configuration

All parameters are validated once in `DvGuidanceConfig::create()` when the adapter's `reset()` or `reconfigure()` runs; the algorithm never sees an invalid configuration.

Table 2: Configuration parameters

Parameter Name	Type	Valid range	Default	Description
`cmdDv_N`	Eigen::Vector3f	finite, $\|\Delta\boldsymbol{v}\| \geq 10^{-6}$ [m/s]	**0** (must be set)	commanded delta-V vector in inertial-frame components; defines the 3rd axis of $\mathcal{R}$
`r2Pref_N`	Eigen::Vector3f	finite, nonzero, at least $\sim 0.06°$ from parallel or antiparallel to `cmdDv_N`	**0** (must be set)	preferred direction of the 2nd axis of $\mathcal{R}$ in inertial-frame components; only its direction is used, and only its component perpendicular to `cmdDv_N` affects the frame

Two-Phase Initialization

The Python usage follows the standard adapter lifecycle: set the public configuration properties, call `reset()` once, then drive `updateState()` each cycle:

```
module = dvGuidanceF32.DvGuidance()
module.cmdDv_N = [5.0, 5.0, 5.0]
module.r2Pref_N = [1.0, 0.0, 0.0]

sim.AddModelToTask(task_name, module)
sim.InitializeSimulation()
sim.ExecuteSimulation()
```

If `cmdDv_N` or `r2Pref_N` is invalid when `reset()` runs, including when either is left at its zero default, an `fsw::invalid_argument` is thrown. If `updateState()` is called before `reset()`, an `XmeraLifecycleException` is thrown. To change the configuration after `reset()`, set the properties and call `reconfigure()`; the new values are used from the next `updateState()`.

Mathematical Formulation

All vector components in this section are taken in a common inertial frame $\mathcal{N}$.

Burn Reference Frame

Let $\mathcal{R} = \{\hat{\boldsymbol{r}}_1, \hat{\boldsymbol{r}}_2, \hat{\boldsymbol{r}}_3\}$ be the burn reference frame. Its DCM relative to $\mathcal{N}$ is

$$[RN] = \begin{bmatrix} \hat{\boldsymbol{r}}_1^T \\ \hat{\boldsymbol{r}}_2^T \\ \hat{\boldsymbol{r}}_3^T \end{bmatrix}.$$

The axes are constructed from the commanded delta-V vector $\Delta\boldsymbol{v}$ (`cmdDv_N`) and the preferred 2nd-axis direction $\boldsymbol{r}_{2,\text{pref}}$ (`r2Pref_N`), normalized to $\hat{\boldsymbol{r}}_{2,\text{pref}}$:

- $\hat{\boldsymbol{r}}_3$ points along the commanded delta-V vector,

$$\hat{\boldsymbol{r}}_3 = \frac{\Delta\boldsymbol{v}}{\|\Delta\boldsymbol{v}\|},$$

- $\hat{\boldsymbol{r}}_1$ is the cross product of the preferred direction with $\hat{\boldsymbol{r}}_3$,

$$\hat{\boldsymbol{r}}_1 = \frac{\hat{\boldsymbol{r}}_{2,\text{pref}} \times \hat{\boldsymbol{r}}_3}{\|\hat{\boldsymbol{r}}_{2,\text{pref}} \times \hat{\boldsymbol{r}}_3\|},$$

- $\hat{\boldsymbol{r}}_2$ completes the right-handed frame,

$$\hat{\boldsymbol{r}}_2 = \hat{\boldsymbol{r}}_3 \times \hat{\boldsymbol{r}}_1.$$

The resulting 2nd axis is the component of $\hat{\boldsymbol{r}}_{2,\text{pref}}$ perpendicular to the commanded delta-V vector, normalized: the direction closest to the preferred direction that is perpendicular to $\Delta\boldsymbol{v}$. When $\hat{\boldsymbol{r}}_{2,\text{pref}}$ is already perpendicular to $\Delta\boldsymbol{v}$, $\hat{\boldsymbol{r}}_2 = \hat{\boldsymbol{r}}_{2,\text{pref}}$. The construction is illustrated in Figure 1.

The MRP output is

$$\boldsymbol{\sigma}_{R/N} = \text{C2MRP}([RN]).$$

Because $\mathcal{R}$ depends only on the configuration, $[RN]$ and $\boldsymbol{\sigma}_{R/N}$ are computed once in `setConfig()`, when a configuration is installed at construction or by `reconfigure()`. Each `update()` returns the stored $\boldsymbol{\sigma}_{R/N}$.

Angular Rate and Acceleration

The burn reference frame depends only on the configuration, so it is inertially fixed between configuration changes. The algorithm does not output rates; the adapter publishes

$$\boldsymbol{\omega}_{R/N} = \mathbf{0}, \qquad \dot{\boldsymbol{\omega}}_{R/N} = \mathbf{0}.$$

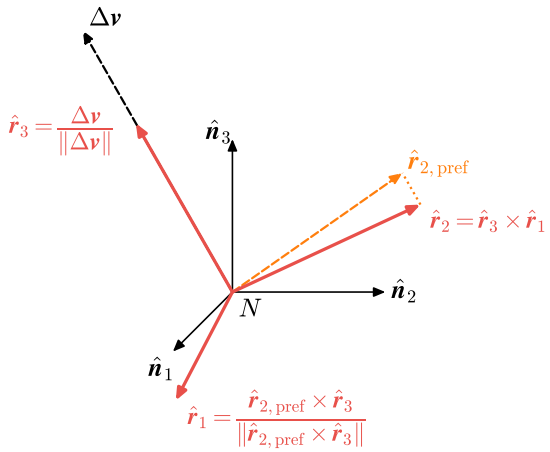


Figure 1: Construction of the burn reference frame $\mathcal{R}$ from the commanded delta-V vector $\Delta \boldsymbol{v}$ and the preferred 2nd-axis direction $\hat{\boldsymbol{r}}_{2,\mathrm{pref}}$. The dotted line is the component of $\hat{\boldsymbol{r}}_{2,\mathrm{pref}}$ along $\Delta \boldsymbol{v}$ that is removed to obtain $\hat{\boldsymbol{r}}_2$.

Configuration Validation

Two degenerate geometries cannot define the burn reference frame, and `DvGuidanceConfig::create()` rejects them by throwing `fsw::invalid_argument`:

- A near-zero $\Delta \boldsymbol{v}$ does not define a burn direction. The configuration is rejected when $\|\Delta \boldsymbol{v}\|$ falls below `kMinNorm`.
- A preferred 2nd-axis direction that is zero, or sufficiently close to parallel or antiparallel with $\hat{\boldsymbol{r}}_3$, collapses the cross product used to construct $\hat{\boldsymbol{r}}_1$. The configuration is rejected when the magnitude of $\hat{\boldsymbol{r}}_{2,\mathrm{pref}} \times \hat{\boldsymbol{r}}_3$ falls below `kMinCross`.

Non-finite values are rejected as well. Because an invalid configuration cannot be constructed, the frame construction has no degenerate-input branch.

Assumptions and Limitations

- The spacecraft body 3rd axis is assumed to be the thrust direction: $\hat{\boldsymbol{r}}_3$ is aligned with $+\Delta \boldsymbol{v}$, so tracking $\mathcal{R}$ produces the commanded delta-V only if the thrust acts along the body $+\hat{\boldsymbol{b}}_3$ axis.
- `r2Pref_N` is a preference, not an enforced axis. The achieved 2nd axis is the projection of `r2Pref_N` onto the plane perpendicular to the commanded delta-V vector, so it differs from `r2Pref_N` by the angle between them; near the `kMinCross` limit the two are almost perpendicular.
- The burn direction is held inertially fixed for the whole burn. A burn direc-

tion that should change during the burn (for example, to follow the orbit) must be provided by updating the configuration through `reconfigure()`.

Numerical conditioning

`kMinCross` = 10^{-3} (an angle of about 0.06°) is the same small-angle threshold used by the other FP32 guidance algorithms that build a frame from a cross product. FP32 round-off in the frame construction grows as `r2Pref_N` approaches `cmdDv_N`, reaching about 10^{-4} at the `kMinCross` limit, so the regression tests compare each DCM element against the double-precision reference with a tolerance of 10^{-4}.

`kMinNorm` = 10^{-6} m/s is not set by FP32 round-off. The frame construction normalizes `cmdDv_N` with a scaled norm, so its direction is accurate for magnitudes far below `kMinNorm`, and the validation check is accurate down to about 10^{-19} m/s, where the squared components underflow. `kMinNorm` is instead a floor below which the commanded delta-V is treated as effectively zero: such a command does not describe a meaningful burn, so its direction is treated as undefined.